FAKULTA
INFORMAČNÍCH
TECHNOLOGIÍ
ČVUT V PRAZE

NI-NLM – Lecture 3

LLM Training

Zdeněk Kasner

3 Mar 2026

Training neural networks

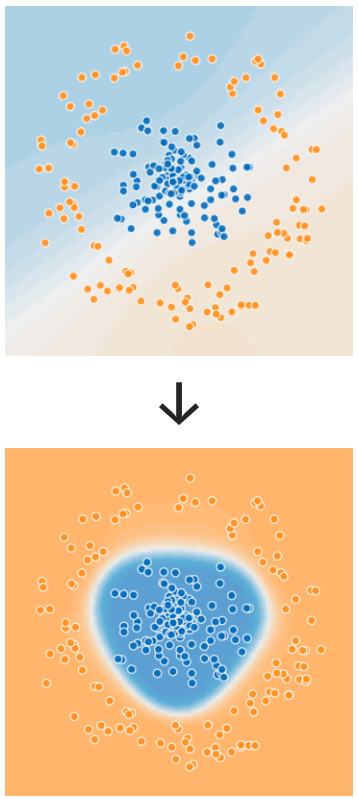
Training neural networks

source: https://playground.tensorflow.org

👩‍🍳 Recipe for supervised training of NNs

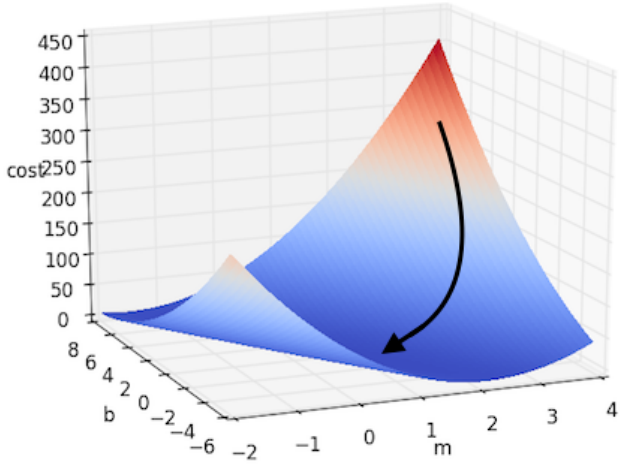
Repeat for each training example (x, y):

1. Let the model **predict** $\mathcal{M}(x, y) = \hat{y}$.
2. Compute a **loss function** $\mathcal{L}(\hat{y}, y)$: how wrong the model prediction $\hat{y}$ is.
3. Compute **gradients**: how each parameter of the model $\mathcal{M}$ contributed to the loss $\mathcal{L}$.
4. Apply **backpropagation**: update the model $\mathcal{M}$ parameters in the direction that reduces the loss $\mathcal{L}$.

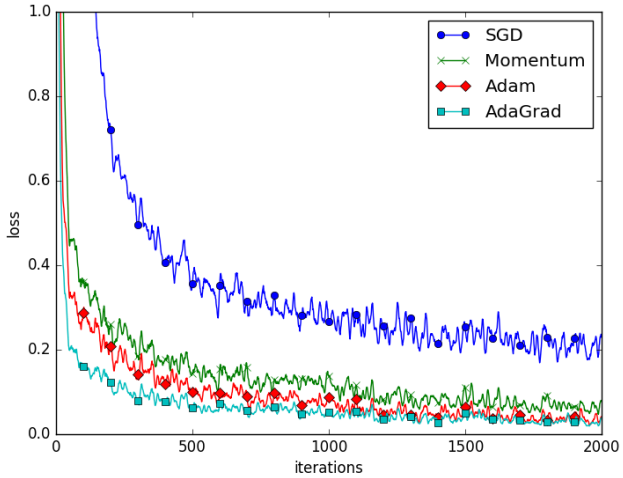


How does training a neural network look like?

source: https://playground.tensorflow.org



source: https://ml4a.github.io



source: https://kaeken.hatenablog.com/

How does training a language model look like?

source: Karpathy's microgpt

```python
for step in range(num_steps):

    # Take single document, tokenize it, surround it with BOS special token on both sides
    doc = docs[step % len(docs)]
    tokens = [BOS] + [uchars.index(ch) for ch in doc] + [BOS]
    n = min(block_size, len(tokens) - 1)

    # Forward the token sequence through the model, building up the computation graph all the way to the loss
    keys, values = [[] for _ in range(n_layer)], [[] for _ in range(n_layer)]
    losses = []
    for pos_id in range(n):
        token_id, target_id = tokens[pos_id], tokens[pos_id + 1]
        logits = gpt(token_id, pos_id, keys, values)
        probs = softmax(logits)
        loss_t = -probs[target_id].log()
        losses.append(loss_t)
    loss = (1 / n) * sum(losses) # final average loss over the document sequence. May yours be low.

    # Backward the loss, calculating the gradients with respect to all model parameters
    loss.backward()

    # Adam optimizer update: update the model parameters based on the corresponding gradients
    lr_t = learning_rate * (1 - step / num_steps) # linear learning rate decay
    for i, p in enumerate(params):
        m[i] = beta1 * m[i] + (1 - beta1) * p.grad
        v[i] = beta2 * v[i] + (1 - beta2) * p.grad ** 2
        m_hat = m[i] / (1 - beta1 ** (step + 1))
        v_hat = v[i] / (1 - beta2 ** (step + 1))
        p.data -= lr_t * m_hat / (v_hat ** 0.5 + eps_adam)
        p.grad = 0
```

Loss function

A **loss function** (or cost function) measures **how far off** the model's prediction is from the correct answer.

How do we measure that? Depends on our problem.

> **Question**
>
> Assume your model predicts a **real number**. How would you measure the error with respect to the ground truth value?

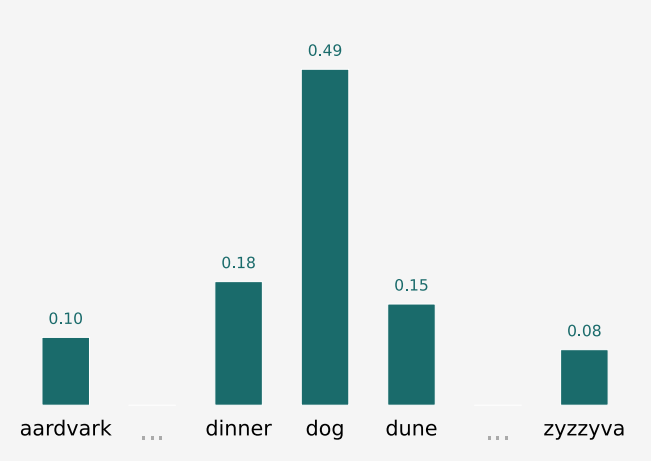
- `(true - predicted)`: bad idea, the errors may cancel out
- `abs(true - predicted)`: ok, but non-differentiable at 0
- `(true - predicted)^2`: works well → smooth, penalizes large errors

Loss function: language modeling

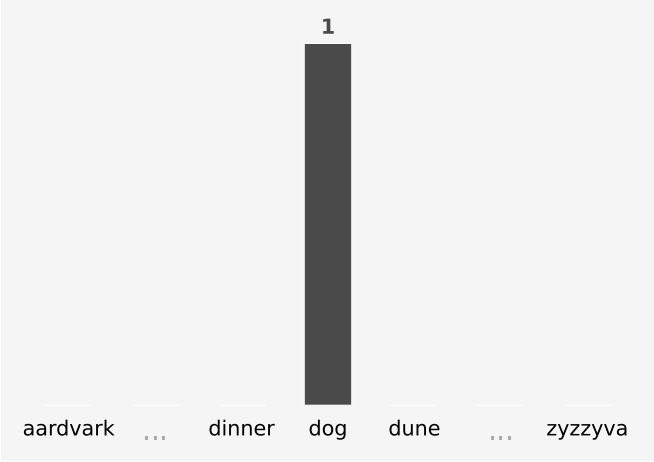
Question

Assume your model predicts a **probability distribution for the next word**. How would you measure the error with respect to the ground truth value?

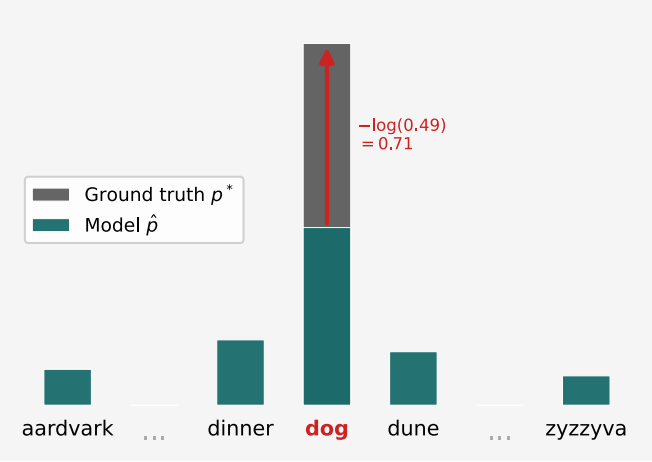
Model prediction



Ground truth



Model error



Cross-entropy loss

The loss function used for the next word prediction task is called **cross-entropy**:

$$\mathcal{L} = -\sum_{c=1}^{C} y_c \cdot \log(\hat{y}_c)$$

where y_c is the true label (one-hot) and $\hat{y}_c$ is the predicted probability.

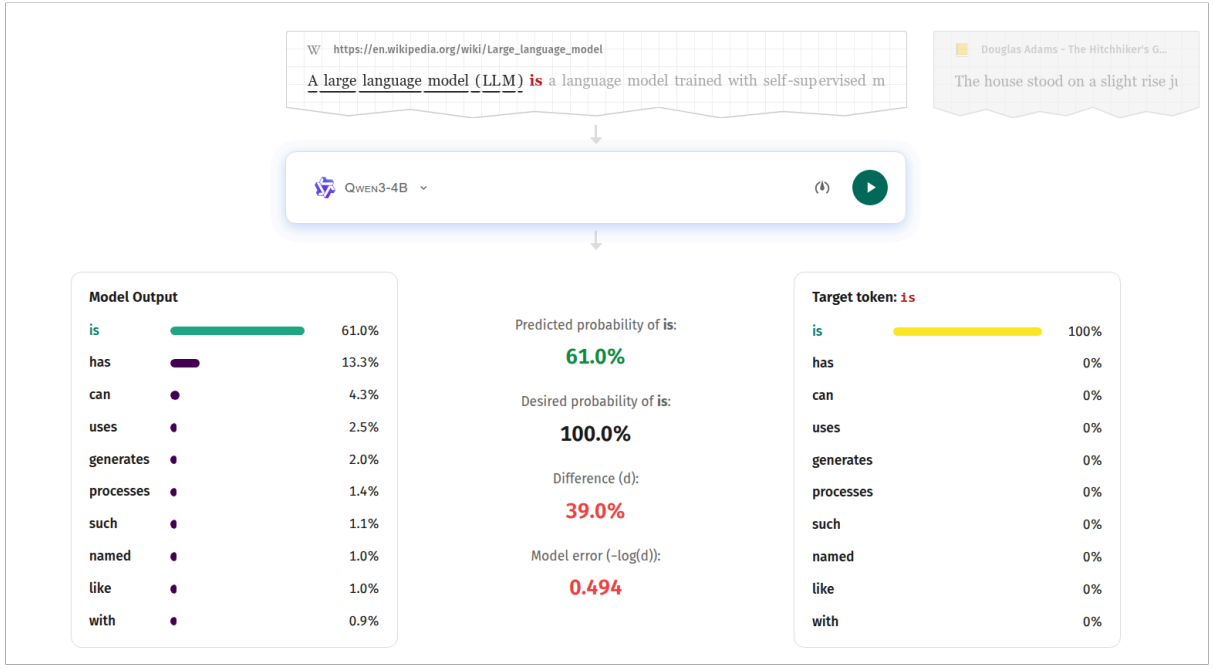
Intuition about cross-entropy for language modeling

- The model is supposed to predict the target token with **100% probability**.
- Any difference between these probabilities is considered as an **error**.
- The value of the loss is a **negative logarithm of the difference** (why?).
- We ignore model predictions for any other tokens.

Cross-entropy in language models

Let's see it animated:

https://animatedllm.github.io/pretraining-simple



Descending the loss landscape

source: Hackernoon

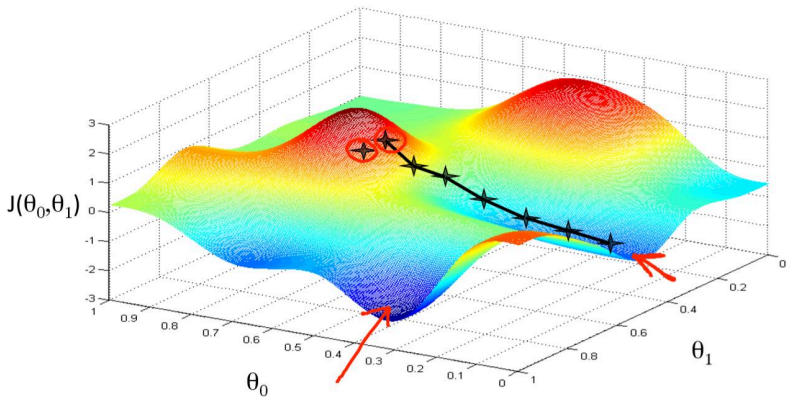
Our goal is to **minimize the loss** by iteratively **updating the model parameters** θ in the direction that reduces the loss $\mathcal{L}$:

Gradient descent as a math formula

$$\theta \leftarrow \theta - \eta \nabla \mathcal{L}(\theta)$$

This simply says: we compute the steepest ascent direction (∇) and go the opossite way.

- ∇ is the **gradient** (derivative) of the loss w.r.t. each parameter.
- η is the **learning rate**.



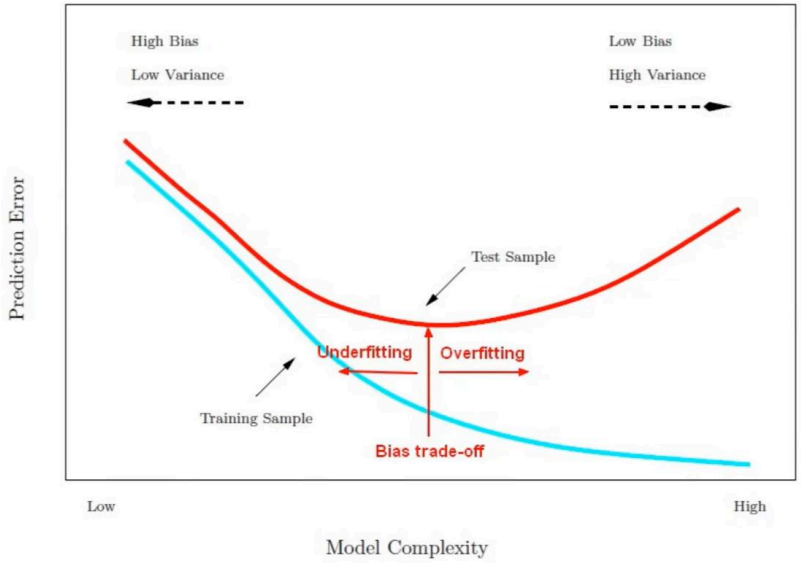
Learning rate

source: CS231n

The **learning rate** η controls how big the parameter update steps are.

Intuition about learning rate

- **Too high** → may overshoot the minimum, training diverges.
- **Too low** → converges very slowly, may get stuck in a bad local minimum.
- It is usually a small multiplier: 0.01, 0.001, …



Computing gradient

Question

We computed the loss: the end point of the forward pass. Now what?

We start by computing $\frac{\partial \mathcal{L}}{\partial p}$ → **the derivative of the loss with respect to the model's output probabilities** (the softmax outputs).

Since $\mathcal{L} = -\log p(\text{correct})$, then $\frac{\partial \mathcal{L}}{\partial p} = -\frac{1}{p(\text{correct})}$ for the correct class and 0 for others.

Gradient vs. derivation

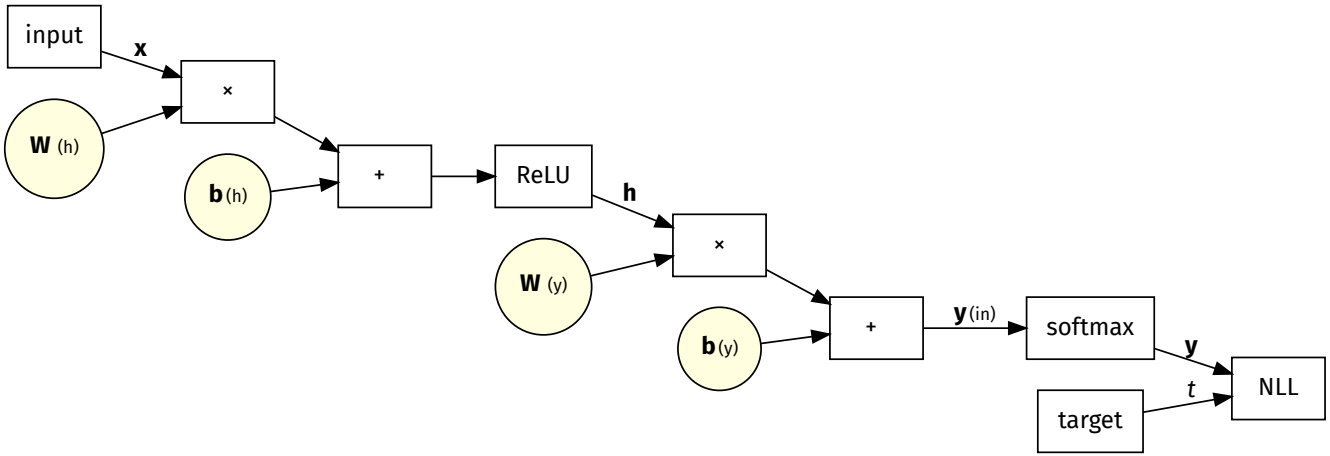
The gradient ∇ is how a (partial) **derivation** ∂ is called for multivariate functions (=functions operating on multi-dimensional inputs).

Backpropagation

Backpropagation = application of the **chain rule** for derivatives for $f(g(x))$:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial g} \cdot \frac{\partial g}{\partial x}$$

How to apply it? Follow the **computation graph** in the backward direction:



Stochastic gradient descent

Computing the gradient over the **entire dataset** is (very, very) expensive.

Idea

Instead of using all training examples, we iteratively compute the gradient on **small random subsets** of the training examples.

- **Stochastic gradient descent (SGD)**: our subset is of size 1 (=a single example).
- **Mini-batch SGD**: we a take a batch of examples (e.g. 32 examples).

Question

Is it useful to introduce some stochasticity into the training process?

Into the depths of deep learning

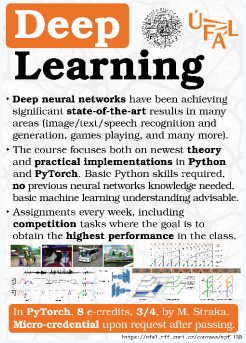
There are other concepts for NN training that we do not cover here, such as:

- **Weight initialization**: how do we initialize the model weights before training?
- **Regularization**: how do we prevent the model from overfitting to training data?
- **Optimizers**: is there are more efficient approach than SGD?

If you want to learn more...

See Milan Straka's **Deep Learning course** (lectures 1-3, lecture recordings available):

https://ufal.mff.cuni.cz/courses/npfl138/2526-summer



Pretraining the Transformer

Self-supervised pretraining

Idea

Train the model on a **huge amount of text data**, without any human-provided labels. Let the model learn language patterns by predicting parts of the text from other parts.

- Also called **self-supervised learning**: the labels come from the text itself.
- Use specific **training objectives** (tasks) that don't need manual annotation.
- Good for learning general language representations on massive data.

Question

> What “tasks” can we create from raw text without human annotations?

Pretraining objectives

source: Devlin et al. 2019 (BERT)

Depending on the architecture, we use different pretraining objectives:

Encoder

Masked language modeling (MLM): mask out some tokens, predict them from the surrounding context.

- E.g. BERT, RoBERTa
- Bidirectional context

Decoder

Causal language modeling (CLM): predict the next token from the previous ones.

- E.g. GPT, GPT-2, LLaMA
- Left-to-right context only

Encoder-decoder

Span corruption / denoising: corrupt spans in the input, reconstruct them in the decoder.

- E.g. T5, BART
- Bidirectional encoder + autoregressive decoder

Causal language modeling (CLM)

This is what we already know as **language modeling** from Lecture 1:

$$P(w_1, ..., w_T) = \prod_{t=1}^{T} P(w_t \mid w_1, ..., w_{t-1})$$

- The model predicts the **next token**, given all previous tokens.
- At training time: the model sees the full sequence, but each position can only attend to **previous** positions (causal masking).
- Loss: cross-entropy at **each position**.

Why causal?

> “Causal” = each prediction depends only on the **past**, not the future. This is what makes it natural for **text generation**.

Causal attention masking

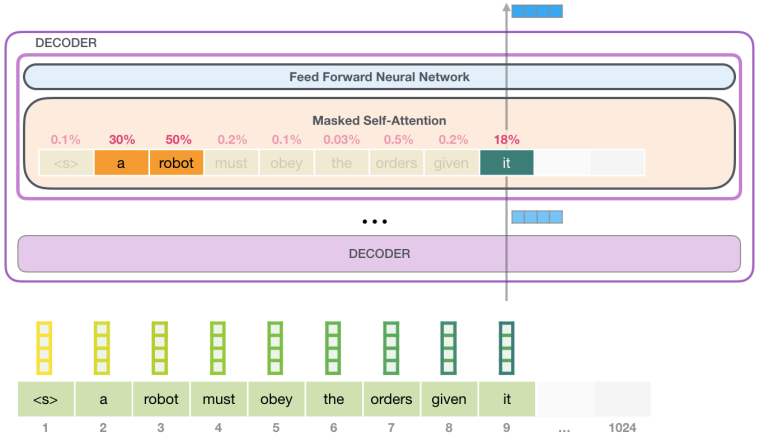
source: The Illustrated GPT-2

In the Transformer decoder, **self-attention is masked**: each token can only attend to itself and earlier tokens.

This is achieved with an **attention mask**:

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V$$

where M is a mask with $-\infty$ in the upper triangle (future positions).



The masking is what allows **parallel training**: we can compute the loss at all positions at once,

even though each position “pretends” it can’t see the future.

Masked language modeling (MLM)

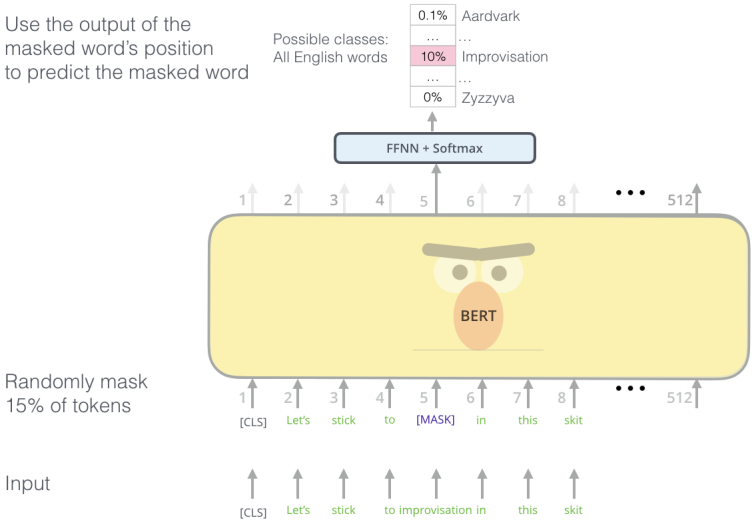
source: Devlin et al. 2019 (BERT)

Instead of predicting the next token, **randomly mask** some tokens and predict them from the full (bidirectional) context.

BERT's masking strategy (15% of tokens):

- 80% replaced with [MASK]
- 10% replaced with a random token
- 10% kept unchanged

Loss is computed **only on the masked positions**.

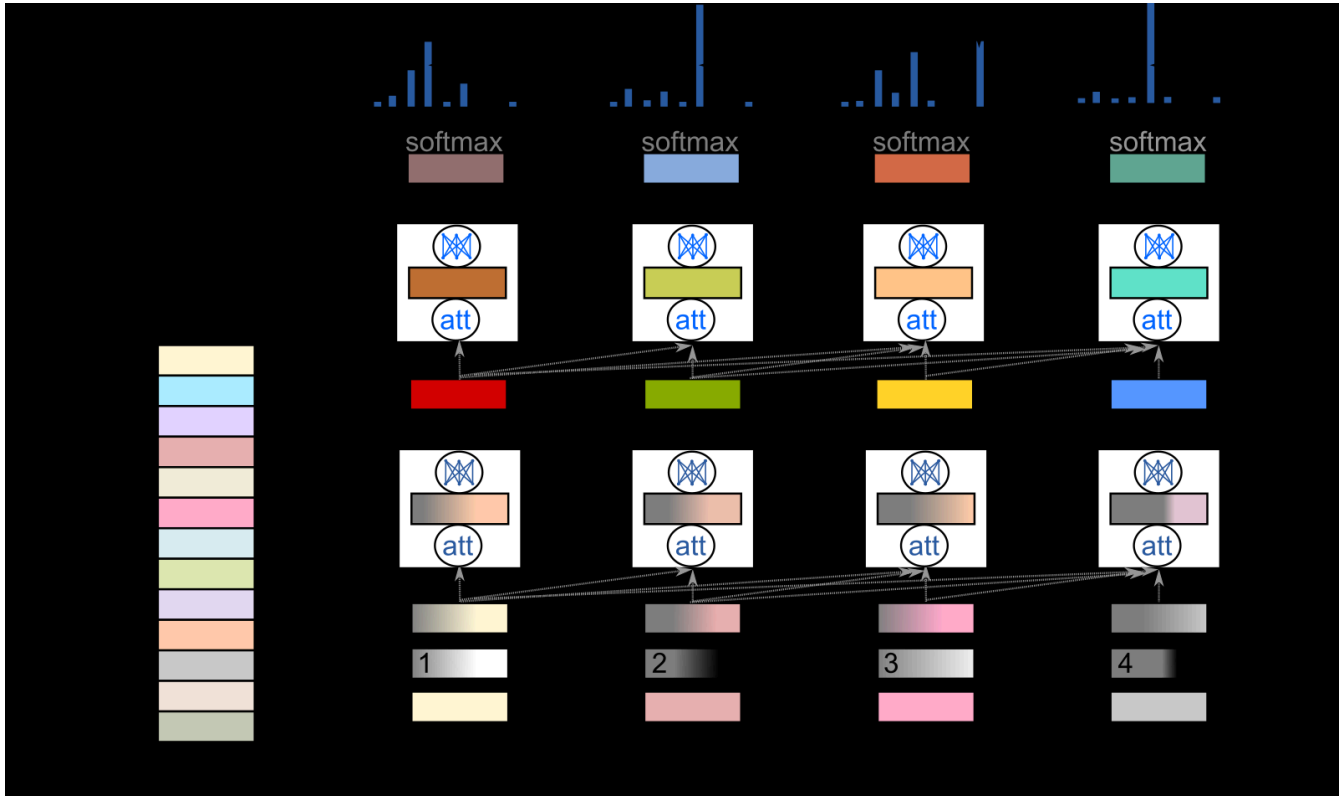


Advantage: the model can use **both left and right context** for prediction.

Training transformers in parallel

source: NPFL140

In the Transformer, we can train on **all positions at once**: feed the entire sequence and predict the next token at each position simultaneously.



This is the key advantage over RNNs, which need sequential processing.

Encoder-decoder pretraining

source: Raffel et al. 2020 (T5)

The encoder-decoder Transformer uses **span corruption** (denoising):

- Replace random **contiguous spans** with sentinel tokens (`<extra_id_0>`, `<extra_id_1>`, ...).
- The decoder generates only the **masked spans** (not the full text).

Example (T5):

Input: `"The <X> brown <Y> over the lazy dog"`

Target: `"<X> quick <Y> fox jumps"`

This combines the benefits of bidirectional encoding with autoregressive generation.

Scaling laws

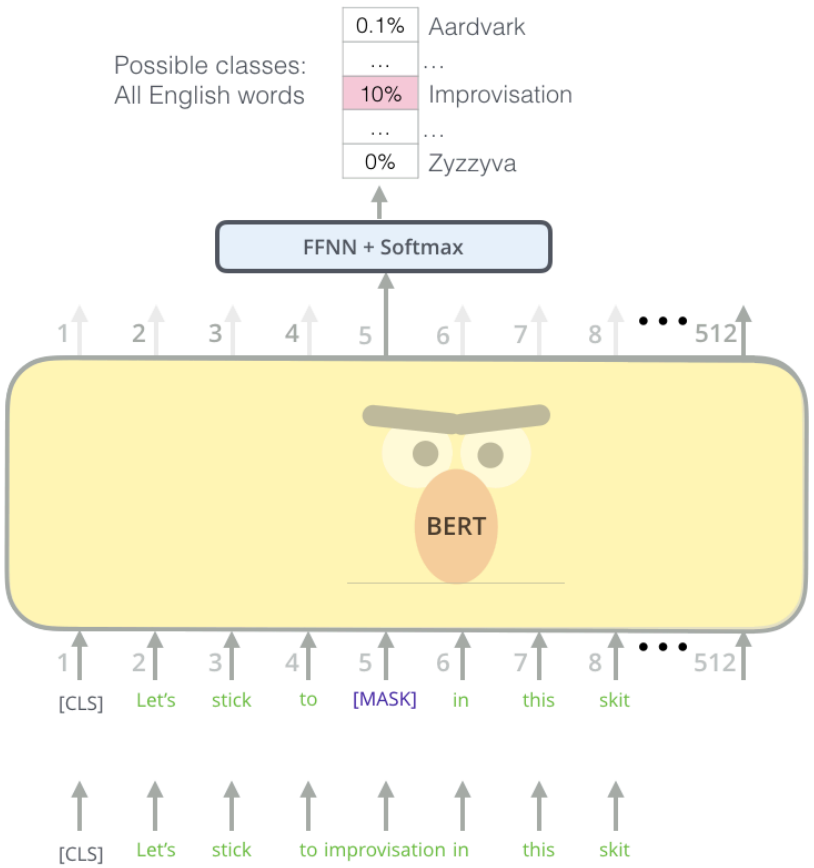
source: Hoffmann et al. 2022 (Chinchilla)

How big should the model and the dataset be?

Chinchilla scaling laws: for a given compute budget, there is an **optimal ratio** of model size to data size.

Rule of thumb: the number of training tokens should be roughly **20× the number of parameters**.

- 7B model → 140B tokens
- 70B model → 1.4T tokens



Many early LLMs were **undertrained** relative to these estimates (e.g. original LLaMA trains on more data → better performance than larger models).

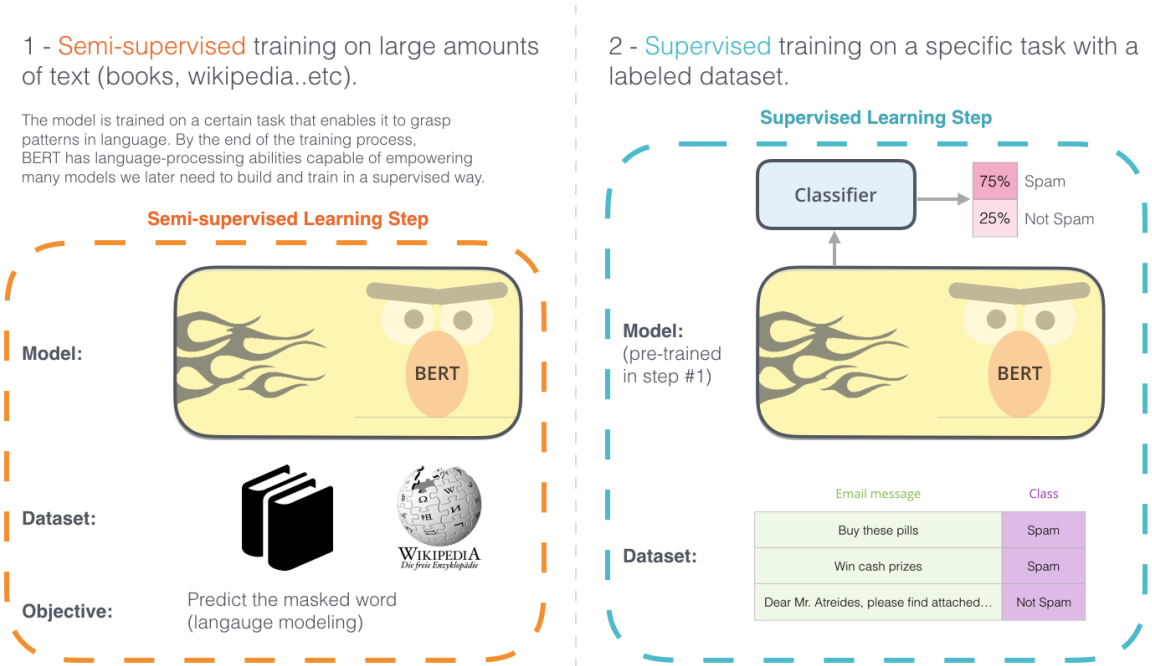
Pretrained models

Pretraining + finetuning paradigm

source: The Illustrated BERT

Two-step training:

1. **Pretrain** a model on a huge dataset (self-supervised, language-based tasks).
2. **Finetune** for your own task on your smaller, task-specific data (supervised).



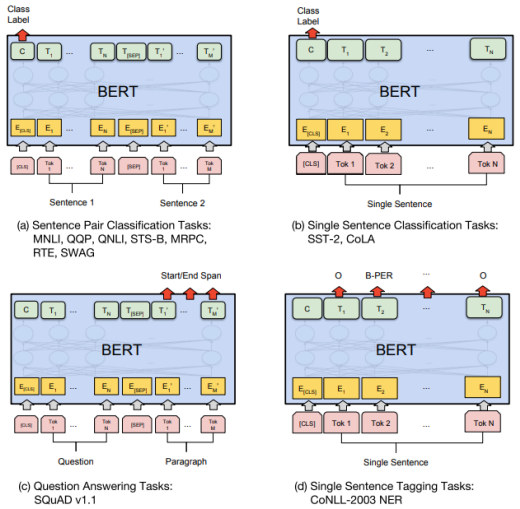
Pretrained models provide **contextual embeddings** – like a better version of word2vec that takes the whole sentence into account.

BERT

source: Devlin et al. 2019

Bidirectional Encoder Representations from Transformers

- Transformer **encoder only**.
- Pretrained with **two objectives**:
  1. Masked language modeling (MLM)
  2. Next sentence prediction (NSP)
- Finetuned by adding a task-specific output layer.
- BERT-base: 12 layers, 768 hidden, 110M params.
- BERT-large: 24 layers, 1024 hidden, 340M params.



BERT set **new SOTA** on 11 NLP tasks when released.

GPT

source: Radford et al. 2018

Generative Pre-trained Transformer

- Transformer **decoder only**.
- Pretrained with **causal language modeling** (next-word prediction).
- GPT-1 (2018): 12 layers, 117M params. Finetuned for downstream tasks.

GPT-2 (2019): scaled up to **1.5B params**, trained on WebText (8M web pages).

source: Radford et al. 2019

Key finding: the model can perform tasks **without any finetuning** – just from the pretraining data. This is what later became known as **zero-shot** capabilities.

GPT-3

source: Brown et al. 2020

175B parameters, trained on a mix of Common Crawl, books, Wikipedia.

Key contribution: **in-context learning** – the model can learn from a few examples provided in the input prompt, without any gradient updates.

Zero-shot

Task description only, no examples.

One-shot

Task description + one example.

Few-shot

Task description + a few examples.

Info

GPT-3 demonstrated that **scale alone** can unlock new capabilities. This was the starting point of the “scaling era” of LLMs.

T5

source: Raffel et al. 2020

Text-to-Text Transfer Transformer

- **Encoder-decoder** architecture.
- All tasks framed as **text-to-text**: input text → output text.

Example:

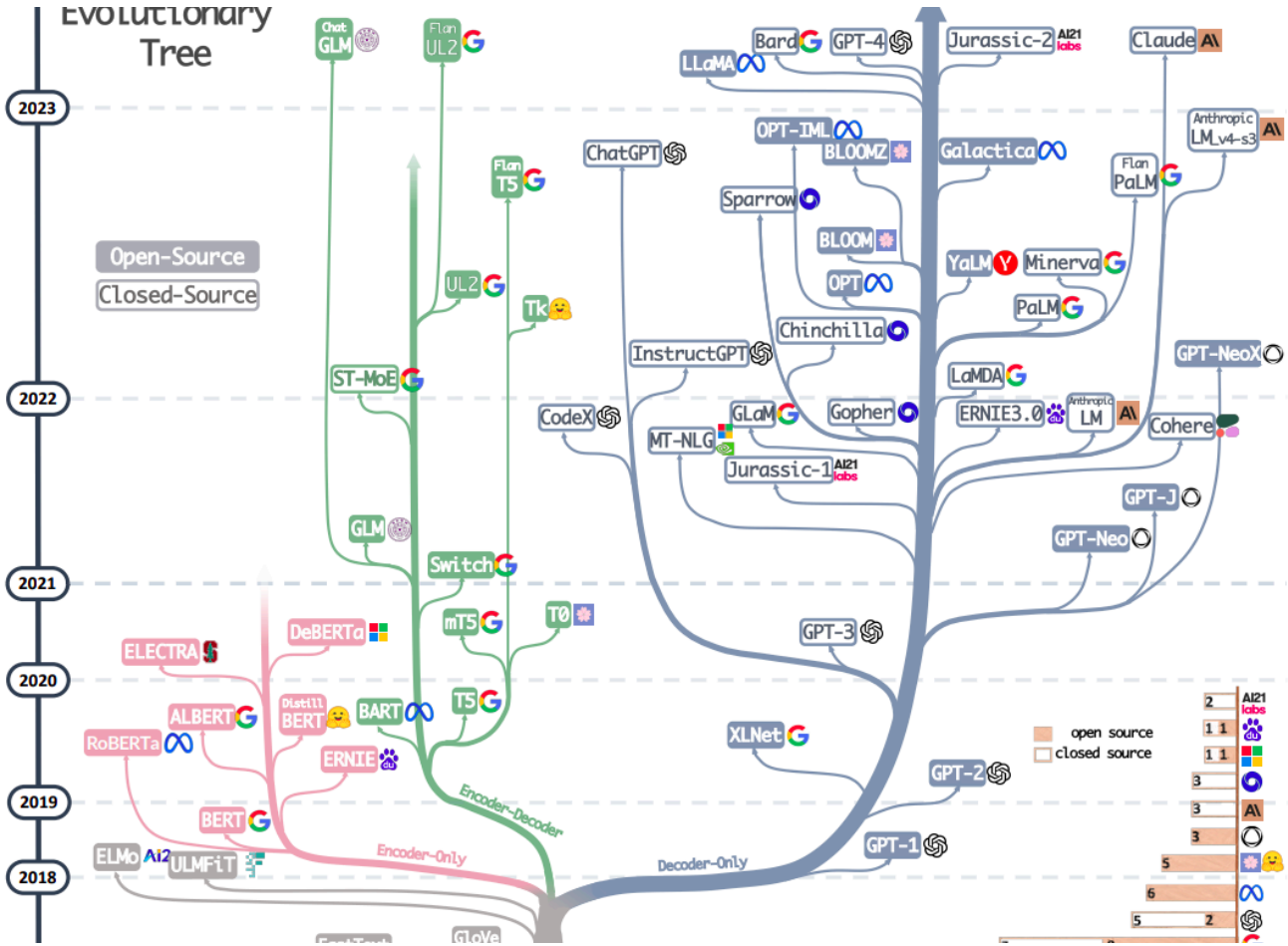
- Translation: `"translate English to German: That is good"` → `"Das ist gut"`
- Summarization: `"summarize: ..."` → summary
- Classification: `"sentiment: great movie"` → `"positive"`

- Trained on C4 (Colossal Clean Crawled Corpus, 750GB of text).

- Various sizes: 60M to 11B parameters.
- Systematically compared many design choices (architecture, objectives, data).

The LLM evolutionary tree

source: Yang et al. 2023



Supervised finetuning

Why finetune?

Pretrained models learn **general language knowledge**. But we often want a model that is good at a **specific task**.

Supervised finetuning (SFT) = continue training the pretrained model on a smaller, task-specific, labeled dataset.

Classic finetuning (BERT era)

- Add a task-specific head (e.g. linear classifier) on top.
- Train on labeled data for the specific task.

Modern finetuning (LLM era)

- Train the model to follow **instructions** in natural language.
- One model for many tasks.
- The task is specified in the prompt.

- Separate model per task.

Finetuning – how it works

1. Start from a **pretrained checkpoint** (e.g. LLaMA-3-8B).
2. Prepare a dataset of **(input, output) pairs** for the target task.
3. Continue training with the **same objective** (cross-entropy on the output tokens).
4. Use a **lower learning rate** than pretraining (the model is already close to a good solution).

Why does this work?

The pretrained model has already learned rich language representations. Finetuning only needs to **slightly adjust** the parameters to fit the new task. This is much more data-efficient than training from scratch.

Instruction tuning

source: Wei et al. 2022 (FLAN)

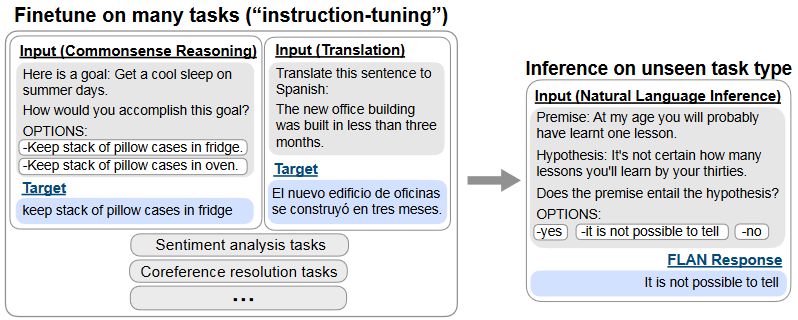
A special form of finetuning: train the model to **follow instructions**.

Training data format:

- **Instruction**: a natural language description of the task.
- **Input**: the actual input for the task (optional).
- **Output**: the expected response.

Example:

- Instruction: "Translate the following sentence to French."



- Input: “How are you?”
- Output: “Comment allez-vous?”

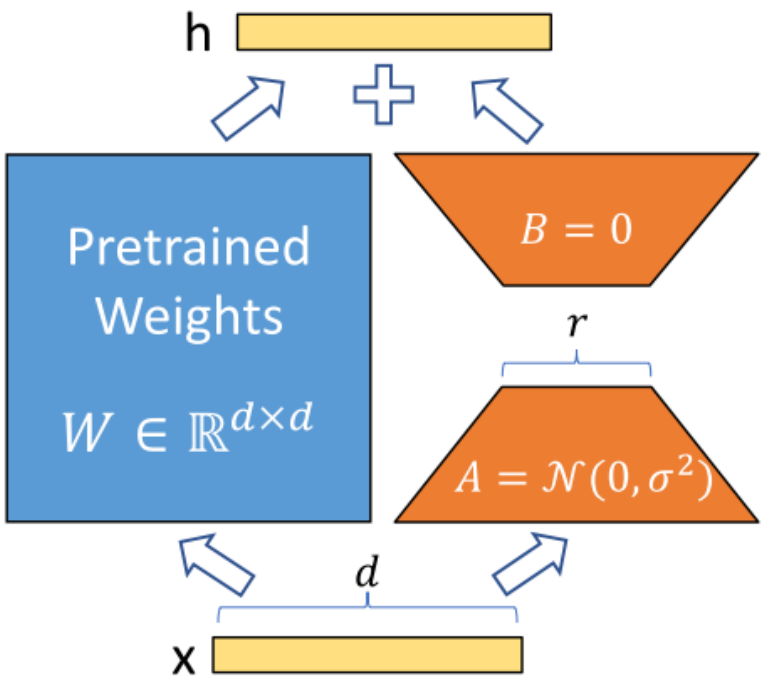
Parameter-efficient finetuning (PEFT)

source: Hu et al. 2021 (LoRA)

Finetuning **all parameters** of a large model is expensive. Can we finetune only **a few**?

LoRA (Low-Rank Adaptation):

- Add two small matrices $A \in \mathbb{R}^{d \times r}$ and $B \in \mathbb{R}^{r \times d}$ parallel to each weight matrix.
- Only train A and B (where $r \ll d$).
- The update $\Delta W = BA$ is added to the original weights.
- Much fewer trainable parameters: $2 \times r \times d$ instead of d^2.



source: Dettmers et al. 2023

QLoRA: LoRA + 4-bit quantization → finetune a 65B model on a single GPU.

Alignment

The alignment problem

A pretrained (or instruction-tuned) model can generate fluent text, but it may still:

- **Hallucinate** facts.
- Produce **harmful or toxic** content.
- Be **unhelpful** or refuse reasonable requests.
- Not follow the user's **intent**.

Alignment

Make the model **helpful, harmless, and honest** – aligned with human preferences.

The key idea: use **human feedback** to teach the model what “good” output looks like, beyond what can be captured by next-token prediction.

LLM training pipeline

The standard training pipeline for modern LLMs has **three stages**:

1. Pretraining

- Self-supervised on massive text data.
- Learn general language knowledge.
- Trillions of tokens.
- Hundreds of GPUs for weeks/months.

2. Supervised finetuning

- Instruction-following data.
- Learn to be helpful and follow instructions.
- Thousands to millions of examples.
- Much cheaper than pretraining.

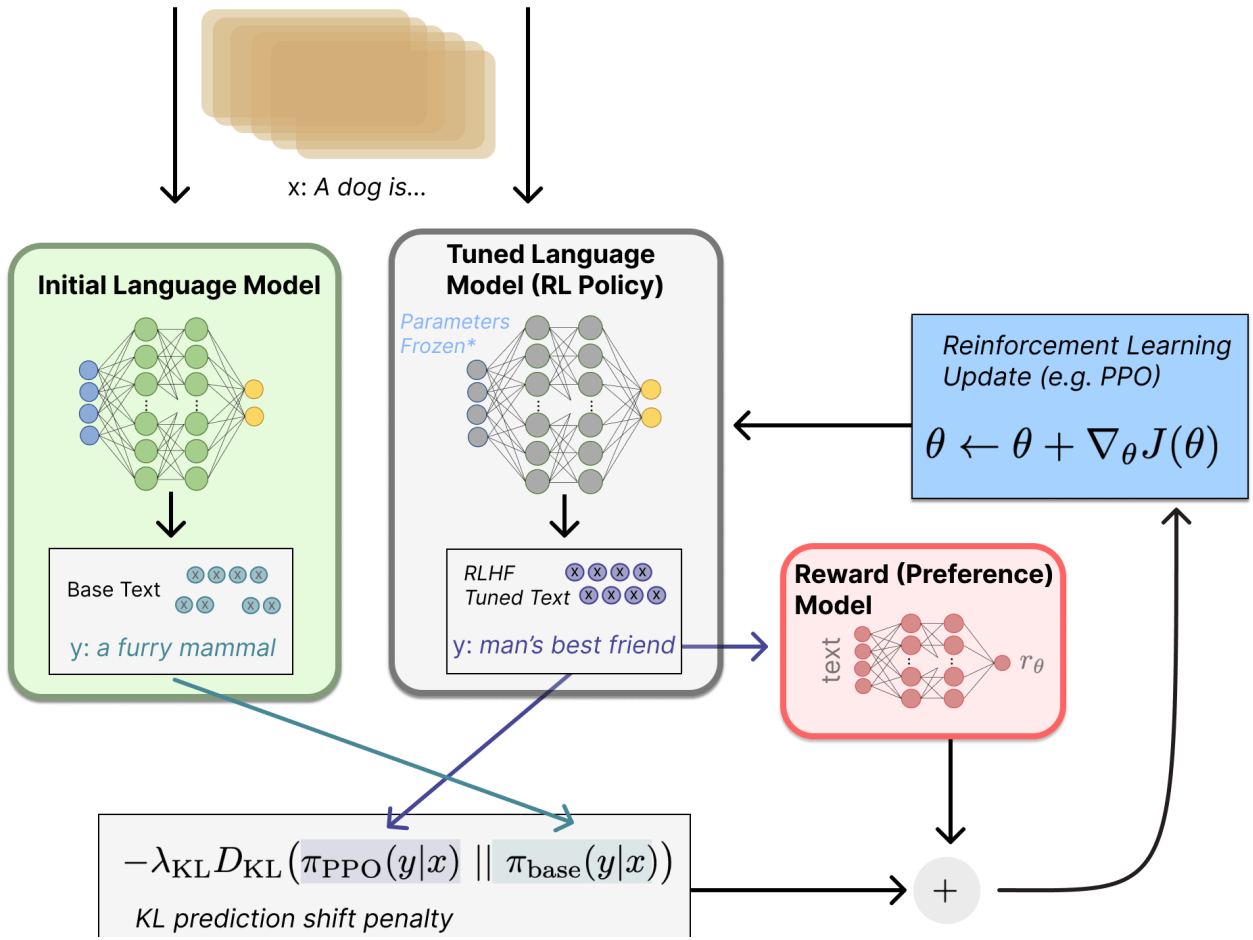
3. Alignment

- Human preference data.
- Learn to produce outputs humans prefer.
- RLHF, DPO, or variants.
- The “secret sauce” of ChatGPT.

Reinforcement learning from human feedback (RLHF)

Source: Ouyang et al. 2022 (InstructGPT)

RLHF was the key technique behind InstructGPT and ChatGPT.


x: A dog is...
Initial Language Model
Base Text
y: a furry mammal
Tuned Language Model (RL Policy)
Parameters Frozen*
RLHF Tuned Text
y: man's best friend
Reinforcement Learning Update (e.g. PPO)
$\theta \leftarrow \theta + \nabla_\theta J(\theta)$
Reward (Preference) Model
text
r_θ
$-\lambda_{\mathrm{KL}} D_{\mathrm{KL}}\left(\pi_{\mathrm{PPO}}(y|x) \;||\; \pi_{\mathrm{base}}(y|x)\right)$
KL prediction shift penalty
+

source: Hugging Face RLHF blog

RLHF – step by step

source: Ouyang et al. 2022

Step 1: Collect comparison data

- Generate multiple outputs for the same prompt.
- Have humans **rank** them (which output is better?).

Step 2: Train a reward model

- A separate model that takes (prompt, response) → score.
- Trained on human preference data.
- Learns to predict which response humans would prefer.

Step 3: Optimize the policy with RL

- Use the reward model as a signal.
- Optimize the LLM to produce outputs that get high reward.
- Usually with **PPO** (Proximal Policy Optimization).
- Include a **KL divergence penalty** to prevent the model from drifting too far from the SFT model.

$$\mathcal{L}_{\text{RLHF}} = -\mathbb{E}[r(x, y)] + \beta \cdot \text{KL}(\pi_\theta \parallel \pi_{\text{ref}})$$

InstructGPT / ChatGPT

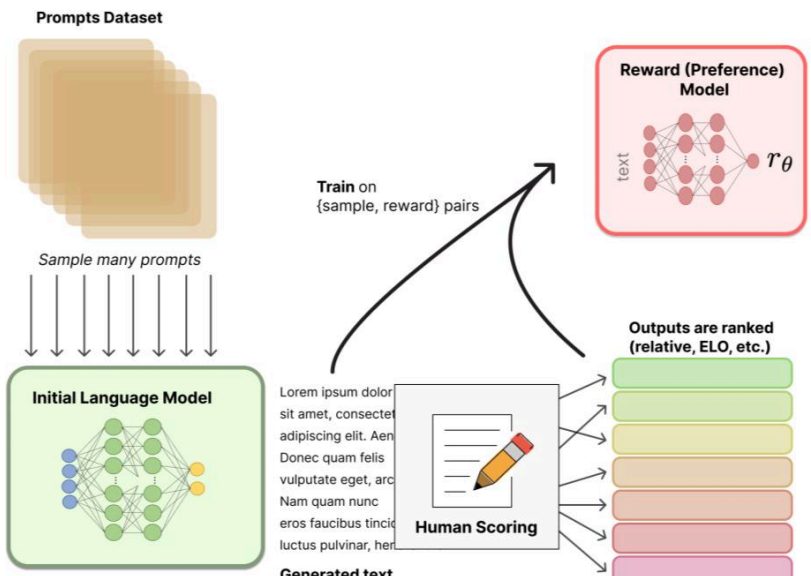
source: Ouyang et al. 2022

InstructGPT = GPT-3 finetuned with RLHF:

1. Start from GPT-3.
2. SFT on human-written demonstrations.
3. Train a reward model on human comparisons.
4. Optimize with PPO using the reward model.

Key result: InstructGPT (1.3B) was **preferred by humans** over the much larger GPT-3 (175B).

ChatGPT (Nov 2022) used the same approach, applied to GPT-3.5.



Direct preference optimization (DPO)

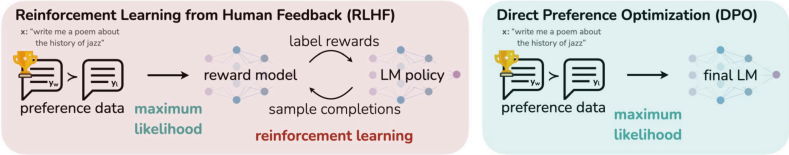
source: Rafailov et al. 2023

RLHF works but is **complex**: three models, unstable PPO, reward hacking. Can we do better?

DPO skips the reward model and RL entirely.

Given pairs of (preferred, dispreferred) responses,
DPO directly optimizes a **special loss function**:

$$\mathcal{L}_{\text{DPO}} = -\log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)$$



where y_w = preferred response, y_l = dispreferred response.

- **Increases** probability of preferred responses.
- **Decreases** probability of dispreferred responses.
- The reference model π_{ref} prevents too large updates.

Summary

Summary

What we covered today

- **Training NNs**: loss functions, gradient descent, SGD, backpropagation, learning rate, optimizers.
- **Pretraining**: self-supervised objectives (CLM, MLM, span corruption).
- **Pretrained models**: BERT, GPT, T5 and their variants; the LLM timeline.
- **Supervised finetuning**: adapting pretrained models to specific tasks; instruction tuning.
- **Alignment**: RLHF, DPO – making models helpful and safe.

Links

- BERT: Pre-training of deep bidirectional Transformers – Devlin et al. (2019)
- Language models are few-shot learners (GPT-3) – Brown et al. (2020)
- Exploring the limits of transfer learning (T5) – Raffel et al. (2020)
- Training language models to follow instructions (InstructGPT) – Ouyang et al. (2022)
- Direct preference optimization (DPO) – Rafailov et al. (2023)
- Illustrating RLHF – Hugging Face blog
- An overview of gradient descent optimization algorithms – Ruder (2016)
- The Illustrated BERT – Jay Alammar